

NLP and classification [Practical MCQ] (Version : 0)

TEST

● **Correct Answer**

🕒 Answered in 4.58 Minutes

Question 1/15

What does the **CountVectorizer** output **X** represent in the code snippet below?

```
from sklearn.feature_extraction.text import CountVectorizer

# Sample text data
data = ["Machine learning is fascinating.", "Natural language processing and machine lea

# Initialise the CountVectorizer
vectorizer = CountVectorizer()

# Fit and transform the data
X = vectorizer.fit_transform(data)

# Get the feature names
feature_names = vectorizer.get_feature_names_out()
```

☐

A list of all words used across the documents.



The frequency of each word in each document.

☐

A binary indication of whether a word appears in a document or not.

☐

The count of unique words in each document.

Explanation:

- **The frequency of each word in each document.** is correct. The matrix **X** is a sparse matrix where each row represents a document and each column represents a word from the entire set of documents, containing the frequency of the word appearances

in each document.

- **The count of unique words in each document**

is incorrect. **CountVectorizer** by default counts the occurrences of words in each document, not just their presence.

- **A list of all words used across the documents.**

is incorrect. It does not provide a list but a sparse matrix representing word frequencies.

- **A binary indication of whether a word appears in a document or not** is incorrect. This would be true if the **binary=True** parameter were used in **CountVectorizer**. By default, it counts occurrences, not just presence.

Question 2/15

Modify the code below to compute and print the accuracy.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4

# Initialise the Logistic Regression model
logreg = LogisticRegression(solver='liblinear')

# Train the model
logreg.fit(X_train, y_train)

# Predict the test set results
y_pred = logreg.predict(X_test)
# insert code here
```

What is the accuracy of the logistic regression model on the test data?

☐ 0.962

☐ 0.975

☐ 0.945

☒ 0.958

Explanation:

- **0.958** is correct. When the code to compute accuracy is added, we obtain an accuracy around 0.958, depending on the specific split of data and model parameters.

- **0.975**, **0.962**, and **0.945** are incorrect. These values might seem plausible but do not match the output generated by the actual computation given the specific random state and solver settings.

Question 3/15

What is the value of True Positive (TP) in the confusion matrix generated by the RandomForestClassifier below? Modify the code to print the value.

```
from sklearn.metrics import confusion_matrix
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Generate synthetic binary classification dataset
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Initialise and train the RandomForestClassifier
rf_classifier = RandomForestClassifier(random_state=42)
rf_classifier.fit(X_train, y_train)

# Predict the test set results
y_pred = rf_classifier.predict(X_test)

# Generate the confusion matrix
cm = confusion_matrix(y_test, y_pred)
# insert code here
```

☐ 97

☐ 118

☒ 113

☐ 130

Explanation:

- **113** is correct. Adding the code snippet to print **cm[1, 1]** will display the number of True Positives, which should closely match this answer depending on the specific details of data generation and model training.

- **118**, **97**, and **130** are incorrect. These values do not match the expected True Positive count for the given random state and classifier settings.

Question 4/15

What is the best value of the parameter 'C' for the SVC according to the grid search? Modify the code to print the best parameter.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
import numpy as np

# Load a dataset
digits = load_digits()
X = digits.data
y = digits.target

# Initialise an SVC (Support Vector Classifier) with a linear kernel
svm = SVC(kernel='linear')

# Define parameter range for C (regularisation parameter)
param_grid = {'C': np.logspace(-3, 3, 7)}

# Setup the grid search with cross-validation
grid_search = GridSearchCV(svm, param_grid, cv=5, scoring='accuracy')

# Fit grid search
grid_search.fit(X, y)
# insert code here
```

☐ 1.0

☒ 0.001

☐ 0.1

 0.01

Explanation:

- A 'C' value of **0.001** provides the best balance between complexity and accuracy for the model, based on the output of **grid_search.best_params_['C']**. A common result for many problems where a moderate amount of regularisation is beneficial is **1.0**. The lower value of 'C' in our result therefore suggests stronger regularisation is optimal for the dataset used.
- The other values are possible options within the range but do not necessarily represent the best regularisation parameter for maximising the accuracy of the SVC on this dataset.

Question 5/15

Which code snippet can be used to fill in the missing lines of code to train the SVM classifier, predict the test set results, and print the classification report?

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report

# Generate a synthetic dataset
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Initialise the SVM classifier with a radial basis function kernel
svm_rbf = SVC(kernel='rbf')

# Fit the classifier to the training data
# [Your Code Here] - Line to add for fitting the model

# Predict the test set results
# [Your Code Here] - Line to add for making predictions

# Generate and print the classification report
# [Your Code Here] - Line to add for printing the classification report
```

 `svm_rbf.fit(X_train, y_train)`
`y_pred = svm_rbf.predict(X_test)`
`print(classification_report(y_test, y_pred))`

☐

```
svm_rbf.fit(X_train, y_train)
y_pred = svm_rbf.predict(X_test)
report = classification_report(y_pred,
y_test)
print(report)
```

☐

```
svm_rbf.train(X_train, y_train)
y_pred = svm_rbf.test(X_test)
print(classification_report(y_pred, y_test))
```

☐

```
svm_rbf.train(X_train, y_train)
y_pred = svm_rbf.classify(X_test)
report = classification_report(y_test,
y_pred)
print(report)
```

Explanation:

- The lines in the correct option rightly implement the standard methods used in **scikit-learn** for fitting a model, predicting outcomes, and printing a classification report which compares predictions with true labels.
- The other options as all incorrect because each of these options includes at least one incorrect method or argument mismatch (like using non-existent methods such as **train**, **classify**, or incorrect order of arguments in the classification report).

Question 6/15

Given the code below, your task is to select the function from the options provided that correctly completes the task by:

- Creating a function that determines which classifier (KNN or Naive Bayes) has a higher F1 score, or if they have equal scores.
- Printing the name of the classifier along with its F1 score in the format: 'ClassifierName has the higher F1 score of Score' or 'Both classifiers have the same F1 score of Score'.
- Executing the function.

Select the appropriate code snippet from the options below.

```

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import f1_score

# Generate a synthetic dataset
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Initialise KNN and Naive Bayes classifiers
knn = KNeighborsClassifier(n_neighbors=5)
nb = GaussianNB()

# Train both classifiers on the training data
knn.fit(X_train, y_train)
nb.fit(X_train, y_train)

# Predict test set results for both classifiers
y_pred_knn = knn.predict(X_test)
y_pred_nb = nb.predict(X_test)

# Calculate F1 scores for both classifiers
f1_knn = f1_score(y_test, y_pred_knn)
f1_nb = f1_score(y_test, y_pred_nb)

# [Your Code Here]

```

☒

```

def print_best_classifier(f1_knn, f1_nb):
    if f1_knn > f1_nb:
        print(f"KNN has the higher F1
              score of {f1_knn}")
    elif f1_knn < f1_nb:
        print(f"Naive Bayes has the higher
              F1 score of {f1_nb}")
    else:
        print(f"Both classifiers have the
              same F1 score of {f1_knn}")
    print_best_classifier(f1_knn, f1_nb)

```

☐

```

def best_f1_score(f1_knn, f1_nb):
    print(f"KNN: {f1_knn}, Naive Bayes:
          {f1_nb}")
    best_f1_score(f1_knn, f1_nb)

```

☐

```

def evaluate_classifiers(f1_knn, f1_nb):
    if f1_knn > f1_nb:
        print(f"Naive Bayes has the higher
              F1 score of {f1_nb}")
    elif f1_nb > f1_knn:
        print(f"KNN has the higher F1

```

```

        score of {f1_knn}")
    else:
        print(f"Both classifiers have the
        same F1 score of {f1_nb}")
    evaluate_classifiers(f1_knn, f1_nb)

```

☐

```

def compare_f1_scores(f1_knn, f1_nb):
    if f1_knn >= f1_nb:
        print(f"KNN has the higher F1
        score of {f1_knn}")
    else:
        print(f"Naive Bayes has the higher
        F1 score of {f1_nb}")
    compare_f1_scores(f1_knn, f1_nb)

```

Explanation:

-

```

def print_best_classifier(f1_knn, f1_nb):

    if f1_knn > f1_nb:
        print(f"KNN has the higher F1 score of
        {f1_knn}")
    elif f1_knn < f1_nb:
        print(f"Naive Bayes has the higher F1
        score of {f1_nb}")
    else:
        print(f"Both classifiers have the same F1
        score of {f1_knn}")
print_best_classifier(f1_knn, f1_nb)

```

Correct because it accurately checks and prints which classifier has a higher F1 score or if they are the same, following the specified format.

-

```

def compare_f1_scores(f1_knn, f1_nb):

    if f1_knn >= f1_nb:
        print(f"KNN has the higher F1 score of
        {f1_knn}")
    else:
        print(f"Naive Bayes has the higher F1
        score of {f1_nb}")
    compare_f1_scores(f1_knn, f1_nb)

```

Incorrect because it assumes KNN is always better or equal, which is misleading and does not handle the equality scenario correctly in terms of printing.

-

```

def best_f1_score(f1_knn, f1_nb):

    print(f"KNN: {f1_knn}, Naive Bayes: {f1_nb}")
best_f1_score(f1_knn, f1_nb)

```


Does not fulfill the requirement to evaluate which classifier is better; it merely prints the scores without comparison.

```
-  
def evaluate_classifiers(f1_knn, f1_nb):  
    if f1_knn > f1_nb:  
        print(f"Naive Bayes has the higher F1  
              score of {f1_nb}")  
    elif f1_nb > f1_knn:  
        print(f"KNN has the higher F1 score of  
              {f1_knn}")  
    else:  
        print(f"Both classifiers have the same F1  
              score of {f1_nb}")  
evaluate_classifiers(f1_knn, f1_nb)
```

Incorrectly swaps the names of the classifiers in the output, providing false information about which classifier performs better.

Question 7/15

Which of the following options will complete the missing code lines to:

- i) train the MLPClassifier,
- ii) predict the test set labels,
- iii) count the number of misclassified samples,
- iv) call the function to print the results.

```

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

# Generate a two-moon dataset
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialise the MLPClassifier with one hidden layer with 10 neurons
mlp = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000, random_state=42)

# [Your Code Here] - Train the MLPClassifier on the scaled training data

# [Your Code Here] - Predict the labels for the scaled test data

# [Your Code Here] - Print the number of misclassified samples in the test set

```

☐

```

mlp.train(X_train_scaled, y_train)
y_pred = mlp.test(X_test_scaled)
print(np.count_nonzero(y_test == y_pred))

```



```

mlp.fit(X_train_scaled, y_train)
y_pred = mlp.predict(X_test_scaled)
print(np.sum(y_test != y_pred))

```

☐

```

mlp.train(X_train_scaled, y_train)
y_pred = mlp.classify(X_test_scaled)
print((y_test - y_pred).count_nonzero())

```

☐

```

mlp.fit(X_train_scaled, y_train)
y_pred = mlp.predict(X_test_scaled)
misclassified = np.where(y_test != y_pred,
1, 0)
print(misclassified.sum())

```

Explanation:

- This correct code snippet accurately trains the MLPClassifier with **fit**, predicts using **predict**, and counts the number of mismatches correctly using **np.sum** on a condition.
- The other snippets either use non-existent

methods (**train**, **classify**, **test**), incorrectly calculate the number of mismatches, or incorrectly sum up the condition.

Question 8/15

Before running the final line of the code in the snippet below to fit the **grid_search** object, you are asked to perform the following tasks directly in the code:

1. Modify the **param_grid** to include a new parameter: **max_features** with values ranging from 1 to 4.
2. Fit the **grid_search** to the training data.
3. After fitting, extract and print the best parameter combination and the corresponding cross-validation score.

Which of the following options correctly completes these tasks?

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Setup a basic decision tree classifier
dt = DecisionTreeClassifier(random_state=42)

# Define a parameter grid over which to optimise the decision tree
param_grid = {
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 10, 20]
}

# Setup the GridSearchCV
grid_search = GridSearchCV(dt, param_grid, cv=5)
```

☒ `param_grid['max_features'] = [1, 2, 3, 4]`
`grid_search = GridSearchCV(dt,`
`param_grid, cv=5)`
`grid_search.fit(X_train, y_train)`
`print(f"Optimal Parameters:`
`{grid_search.best_params_}, CV`
`Accuracy: {grid_search.best_score_}")`

☐ `param_grid.update({'max_features': [1, 2]})`
`grid_search.fit(X_train, y_train)`
`best_params = grid_search.best_params_`
`print(f"Best parameters found:`

```
{best_params}, Score:
{grid_search.best_score_}")
```

```
param_grid = {'max_features': [1, 2, 3, 4]}
grid_search.fit(X_train, y_train)
☐ print("Best Parameters:",
grid_search.best_params_)
print("Best Cross-validation Score:",
grid_search.best_score_)
```

```
☒ param_grid['max_features'] = range(1, 5)
grid_search.fit(X_train, y_train)
print(f"Best Params:
{grid_search.best_params_}, CV Score:
{grid_search.best_score_}")
```

Explanation:

```
-
param_grid['max_features'] = range(1, 5)
grid_search.fit(X_train, y_train)
print(f"Best Params: {grid_search.best_params_},
CV Score: {grid_search.best_score_}")
```

The correct option efficiently updates the `param\_grid`, fits the model, and then directly prints the best parameters and the cross-validation score in a succinct and clear format.

```
-
param_grid.update({'max_features': [1, 2]})
grid_search.fit(X_train, y_train)
best_params = grid_search.best_params_
print(f"Best parameters found: {best_params},
Score: {grid_search.best_score_}")
```

Correct in execution, but verbose. Using `update()` is unnecessary for adding a single parameter. The limited range `[1, 2]` may restrict the exploration of optimal settings.

```
-
param_grid = {'max_features': [1, 2, 3, 4]}
grid_search.fit(X_train, y_train)
print("Best Parameters:",
grid_search.best_params_)
print("Best Cross-validation Score:",
grid_search.best_score_)
```

Incorrect. This redefines the `param\_grid` entirely, losing previous parameters.

```
-  
param_grid['max_features'] = [1, 2, 3, 4]  
grid_search = GridSearchCV(dt, param_grid, cv=5)  
grid_search.fit(X_train, y_train)  
print(f"Optimal Parameters:  
{grid_search.best_params_}, CV Accuracy:  
{grid_search.best_score_}")
```

Incorrect as it unnecessarily reinitializes the
`GridSearchCV`, which is redundant and not
efficient coding practice.

Question 9/15

You are fine-tuning a decision tree classifier for a marketing dataset. To prevent overfitting and ensure robust generalisability, you must adjust the depth of the decision tree after its initialisation but before it is fitted with data. Considering the decision tree `dt` has already been initialised with a random state, which of the following is the correct way to modify the tree's maximum depth?

```
from sklearn.tree import DecisionTreeClassifier  
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
  
# Load data  
data = load_breast_cancer()  
X = data.data  
y = data.target  
  
# Split data  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)  
  
# Initialise decision tree classifier  
dt = DecisionTreeClassifier(random_state=42)  
  
# [Your Code Here
```

☐

dt.set_params(max_depth=5).fit(X_train,
y_train)

☐

dt.max_depth = 42

☐

dt = DecisionTreeClassifier(max_depth=5,
random_state=42)



dt.set_params(max_depth=5)

Explanation:

- **dt.set_params(max_depth=5)**

Correct. This method updates the **max_depth** parameter appropriately without needing to re-initialise the classifier.

- **dt = DecisionTreeClassifier(max_depth=5, random_state=42)**

Incorrect. This line re-initialises the classifier, which is not needed as **dt** has already been initialised.

- **dt.max_depth = 42**

Incorrect. Direct assignment to **max_depth** is ineffective because it does not reconfigure the classifier's parameters officially.

- **dt.set_params(max_depth=5).fit(X_train, y_train)**

Incorrect. Although this sets the parameter and fits the model, the question specifies adjusting the depth before fitting, making this option partially correct but exceeding the required action.

Question 10/15

Suppose you are analysing the performance of a new email spam detection system using precision and recall. You have already computed these metrics, and you are about to explore their trade-offs to optimise the classifier's threshold. Given the code snippet below, identify the correct function call that would allow you to adjust and visualise the precision-recall trade-off.

```
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification

# Generate synthetic data for binary classification
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train a RandomForest classifier
classifier = RandomForestClassifier(random_state=42)
classifier.fit(X_train, y_train)

# Predict probabilities for the test set
y_scores = classifier.predict_proba(X_test)[:, 1]

# [Your Code Here] - Generate precision and recall values for various thresholds
```



precision, recall, thresholds =
`precision_recall_curve(y_test, y_scores)`



precision, recall =
`precision_recall_curve(y_test, y_scores)`



`precision_recall_curve(classifier, X_test,
y_test)`



`plt.plot(precision_recall_curve(y_test,
y_scores))`

Explanation:

- **precision, recall, thresholds =
precision_recall_curve(y_test, y_scores)**

Correct. This function call correctly uses the **precision_recall_curve** function to compute precision and recall for various threshold values, using the probability scores from the classifier.

- **precision_recall_curve(classifier, X_test,
y_test)**

Incorrect. This is a syntactically incorrect use of the **precision_recall_curve** function.

- **plt.plot(precision_recall_curve(y_test,
y_scores))**

Incorrect. While this attempts to plot the precision-recall curve, it misuses the function in the plotting context and lacks variable assignment.

- **precision, recall =
precision_recall_curve(y_test, y_scores)**

Incorrect. This option omits the crucial 'thresholds' output which is typically used to analyse the trade-off between precision and recall.

You are tasked with enhancing the robustness of a logistic regression model by incorporating feature scaling. You're currently working with a dataset that has significantly varying scales among its features, which can affect the model's performance. Below is a preliminary setup for the logistic regression model. Identify the correct sequence of steps to integrate feature scaling into the modelling process.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Initialise the Logistic Regression model
lr = LogisticRegression()

# [Your Code Here] - Apply feature scaling to the training data
# [Your Code Here] - Fit the model on the scaled training data
# [Your Code Here] - Apply the same scaling to the test data
```

☐

```
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
lr.fit(X_train_scaled, y_train)
scaler = StandardScaler()
X_test_scaled =
scaler.fit_transform(X_test)
```

☐

```
scaler = StandardScaler()
X_train_scaled = scaler.transform(X_train)
lr.fit(X_train_scaled, y_train)
X_test_scaled =
scaler.fit_transform(X_test)
```

☐

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
lr.fit(X_scaled, y)
```

☒

```
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
lr.fit(X_train_scaled, y_train)
```

Explanation:

-

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)  
lr.fit(X_train_scaled, y_train)
```

Correct. This sequence properly fits the scaler on the training data, applies the transformation to both training and test datasets in the correct order, and fits the logistic regression model on the scaled training data, avoiding data leakage and ensuring consistent preprocessing.

-

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
lr.fit(X_train_scaled, y_train)  
scaler = StandardScaler()  
X_test_scaled = scaler.fit_transform(X_test)
```

Incorrect. This option initially seems correct as it scales and fits the model on the training data. However, the error of reinitialising and refitting the scaler on the test data violates the principle of using the same scaler for both training and testing, which can mislead and degrade model performance.

-

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)  
lr.fit(X_scaled, y)
```

Incorrect. This approach incorrectly applies scaling to the entire dataset before the train-test split, leading to potential data leakage.

-

```
scaler = StandardScaler()  
X_train_scaled = scaler.transform(X_train)  
lr.fit(X_train_scaled, y_train)  
X_test_scaled = scaler.fit_transform(X_test)
```

Incorrect. This option incorrectly applies transformation and fitting: it doesn't fit the scaler on the training data before transforming it and erroneously refits the scaler to the test data.

Question 12/15

You are fine-tuning a support vector machine (SVM) classifier to categorise images based on their content. The

dataset consists of various animal images, and you suspect that different kernel functions might yield better classification accuracy. You decide to test which SVM kernel—linear or radial basis function (RBF)—works best for your specific dataset. Below is your initial code setup:

```
from sklearn.svm import SVC
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load a dataset of digit images
digits = load_digits()
X = digits.data
y = digits.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Initialise two SVM classifiers, one with a linear kernel and another with an RBF kernel
svm_linear = SVC(kernel='linear')
svm_rbf = SVC(kernel='rbf')

# [Your Code Here] - Train both classifiers on the training data
# [Your Code Here] - Predict the test set results with both classifiers
# [Your Code Here] - Calculate and print the accuracy scores for both classifiers
```

Which of the following options correctly completes the task of training both SVM classifiers, predicting the test set results, and calculating the accuracy for each

☐ `svm_linear.train(X_train, y_train)`
`svm_rbf.train(X_train, y_train)`
`y_pred_linear =`
`svm_linear.classify(X_test)`
`y_pred_rbf = svm_rbf.classify(X_test)`
`print("Linear Kernel Accuracy:",`
`accuracy_score(y_test, y_pred_linear))`
`print("RBF Kernel Accuracy:",`
`accuracy_score(y_test, y_pred_rbf))`

☒ `svm_linear.fit(X_train, y_train)`
`svm_rbf.fit(X_train, y_train)`
`y_pred_linear = svm_linear.predict(X_test)`
`y_pred_rbf = svm_rbf.predict(X_test)`
`print("Linear Accuracy:",`
`accuracy_score(y_test, y_pred_linear))`
`print("RBF Accuracy:",`
`accuracy_score(y_test, y_pred_rbf))`

☐ `svm_linear.fit(X_train, y_train)`
`y_pred_linear =`
`svm_linear.predict(X_train)`
`svm_rbf.fit(X_train, y_train)`
`y_pred_rbf = svm_rbf.predict(X_train)`
`print("Accuracy with Linear Kernel:",`
`accuracy_score(y_train, y_pred_linear))`

```
print("Accuracy with RBF Kernel:",  
      accuracy_score(y_train, y_pred_rbf))
```

```
svm_linear.fit(X_train, y_train)  
y_pred_linear = svm_linear.predict(X_test)  
svm_rbf.fit(X_train, y_train)  
☐ y_pred_rbf = svm_rbf.predict(X_test)  
print("Accuracy with Linear Kernel:",  
      accuracy_score(y_test, y_pred_linear))  
print("Accuracy with RBF Kernel:",  
      accuracy_score(y_test, y_pred_rbf))
```

Explanation:

-

```
svm_linear.fit(X_train, y_train)  
svm_rbf.fit(X_train, y_train)  
y_pred_linear = svm_linear.predict(X_test)  
y_pred_rbf = svm_rbf.predict(X_test)  
print("Linear Accuracy:",  
      accuracy_score(y_test, y_pred_linear))  
print("RBF Accuracy:", accuracy_score(y_test,  
                                       y_pred_rbf))
```

Correct. This option accurately uses the **fit** and **predict** methods on both SVM classifiers and then correctly computes and prints the accuracy scores for each kernel type.

-

```
svm_linear.train(X_train, y_train)  
svm_rbf.train(X_train, y_train)  
y_pred_linear = svm_linear.classify(X_test)  
y_pred_rbf = svm_rbf.classify(X_test)  
print("Linear Kernel Accuracy:",  
      accuracy_score(y_test, y_pred_linear))  
print("RBF Kernel Accuracy:",  
      accuracy_score(y_test, y_pred_rbf))
```

```
svm_linear.fit(X_train, y_train)  
svm_rbf.fit(X_train, y_train)  
y_pred_linear = svm_linear.test(X_test)  
y_pred_rbf = svm_rbf.test(X_test)  
print("Linear Test Accuracy:",  
      accuracy_score(y_test, y_pred_linear))  
print("RBF Test Accuracy:",  
      accuracy_score(y_test, y_pred_rbf))
```

Both Incorrect. They use non-existent **train**, **classify**, and **test** methods which are not part of the sklearn's SVM API.

```
-  
svm_linear.fit(X_train, y_train)  
y_pred_linear = svm_linear.predict(X_train)  
svm_rbf.fit(X_train, y_train)  
y_pred_rbf = svm_rbf.predict(X_train)  
print("Accuracy with Linear Kernel:",  
      accuracy_score(y_train, y_pred_linear))  
print("Accuracy with RBF Kernel:",  
      accuracy_score(y_train, y_pred_rbf))
```

Incorrect. It incorrectly uses training data for prediction and accuracy assessment instead of test data, which is a fundamental error in the context of model evaluation.

Question 13/15

You are currently evaluating two classifiers, K-Nearest Neighbours (KNN) and Naive Bayes, for a project that involves classifying texts into different categories based on their content. To finalise your model selection, you decide to visually compare their performance using a bar chart. Below is the setup for calculating the accuracy of both models on your dataset. Complete the code by adding the necessary lines to plot the accuracies in a bar chart:

```

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load data
data = fetch_20newsgroups(subset='all')
X = data.data
y = data.target

# Create train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Vectorise text data
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Initialise classifiers
knn = KNeighborsClassifier()
nb = MultinomialNB()

# Train classifiers
knn.fit(X_train_tfidf, y_train)
nb.fit(X_train_tfidf, y_train)

# Predict and calculate accuracy
knn_accuracy = accuracy_score(y_test, knn.predict(X_test_tfidf))
nb_accuracy = accuracy_score(y_test, nb.predict(X_test_tfidf))

# [Your code here] - Plot the accuracies in a bar chart

```

Which snippet of code will correctly plot the accuracies of KNN and Naive Bayes classifiers in a bar chart?

☐

```

plt.plot(['KNN', 'Naive Bayes'],
[knn_accuracy, nb_accuracy])
plt.xlabel('Classifier')
plt.ylabel('Accuracy')
plt.title('Comparison of Classifier
Performance')
plt.show()

```

☐

```

plt.bar(['KNN', 'Naive Bayes'],
[knn_accuracy, nb_accuracy])
plt.xlabel('Accuracy')
plt.ylabel('Classifier')
plt.title('Classifier Accuracy Comparison')
plt.show()

```

☒

```

plt.bar(['KNN', 'Naive Bayes'],
[knn_accuracy, nb_accuracy])

```



```
plt.xlabel('Classifier')
plt.ylabel('Accuracy')
plt.title('Classifier Accuracies')
plt.show()
```



```
acc_data = [knn_accuracy, nb_accuracy]
labels = ['KNN', 'Naive Bayes']
plt.barh(labels, acc_data)
plt.xlabel('Accuracy')
plt.ylabel('Classifier')
plt.title('Accuracy Comparison')
plt.show()
```

Explanation:

-

```
plt.bar(['KNN', 'Naive Bayes'], [knn_accuracy,
nb_accuracy])
plt.xlabel('Classifier')
plt.ylabel('Accuracy')
plt.title('Classifier Accuracies')
plt.show()
```

Correct. This option properly uses the **plt.bar** function to create a bar chart displaying the accuracies of the KNN and Naive Bayes classifiers, with appropriate labels and titles.

-

```
plt.plot(['KNN', 'Naive Bayes'], [knn_accuracy,
nb_accuracy])
plt.xlabel('Classifier')
plt.ylabel('Accuracy')
plt.title('Comparison of Classifier Performance')
plt.show()
```

Incorrect. It uses **plt.plot**, which generates a line graph instead of a bar chart.

-

```
acc_data = [knn_accuracy, nb_accuracy]
labels = ['KNN', 'Naive Bayes']
plt.barh(labels, acc_data)
plt.xlabel('Accuracy')
plt.ylabel('Classifier')
plt.title('Accuracy Comparison')
plt.show()
```

Incorrect. It uses **plt.barh** for a horizontal bar chart, which was not specified in the task.

```
plt.bar(['KNN', 'Naive Bayes'], [knn_accuracy,
nb_accuracy])
plt.xlabel('Accuracy')
plt.ylabel('Classifier')
plt.title('Classifier Accuracy Comparison')
plt.show()
```

Incorrect. The axis labels are swapped, which could lead to confusion in interpreting the chart.

Question 14/15

You are tasked with evaluating a simple binary classification model using a confusion matrix. The dataset involves predicting whether a given email is spam or not. To better understand the model's performance, you plan to extract specific metrics from the confusion matrix, specifically True Positives (TP) and False Positives (FP). Below is your initial code setup:

```
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

# Generate synthetic binary classification data
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Train a Random Forest classifier
classifier = RandomForestClassifier(random_state=42)
classifier.fit(X_train, y_train)

# Predict the test set results
y_pred = classifier.predict(X_test)

# Generate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# [Your code here] - Extract and print True Positives and False Positives
```

Which snippet of code correctly extracts and prints the True Positives (TP) and False Positives (FP) from the confusion matrix?

Which snippet of code correctly completes the setup to create a pipeline including **PolynomialFeatures** and **LogisticRegression**, fits it on the training data, and makes predictions?

☐ print("True Positives:", cm[2][2])
print("False Positives:", cm[1][2])

☐ print("TP:", cm[2, 2])

☐ `print("FP:", cm[1, 2])`

☒ `tp = cm[1, 1]`
`fp = cm[0, 1]`
`print("True Positives:", tp)`
`print("False Positives:", fp)`

☐ `print("TP:", cm[1][1])`
`print("FP:", cm[2][1])`

Explanation:

- The right option correctly identifies TP and FP from the confusion matrix where TP is at `cm[1, 1]` (second row, second column) and FP is at `cm[0, 1]` (first row, second column), which are the standard positions in a confusion matrix for binary classification.
- The other options attempt to access indices that are out of bounds for a typical 2x2 confusion matrix used in binary classification, or they attempt to use indices that do not exist, leading to an error or incorrect data extraction. These answers are clearly incorrect with no ambiguity or overlap with correct coding practices.

Question 15/15

You are refining a logistic regression model to predict customer churn. The dataset includes various customer interaction metrics. To enhance your model, explore how polynomial features can improve prediction accuracy. This approach allows the model to capture complex interactions between variables.

Here is your setup:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import PolynomialFeatures

# Generate synthetic data for binary classification
X, y = make_classification(n_samples=1000, n_features=3, n_classes=2, random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=4)

# Apply polynomial features manually
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```


What is the correct procedure to fit a logistic regression model on the training data after transforming it with polynomial features, and how should predictions be made on the test data?

☐ `model = LogisticRegression()
model.fit(X_test_poly, y_test)
y_pred = model.predict(X_train_poly)`

☒ `model = LogisticRegression()
model.fit(X_train_poly, y_train)
y_pred = model.predict(X_test_poly)`

☐ `model = LogisticRegression()
model.fit(X_train_poly, y_test)
y_pred = model.predict(X_test_poly)`

☐ `model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)`

Explanation:

-

**`model = LogisticRegression()
model.fit(X_train_poly, y_train)
y_pred = model.predict(X_test_poly)`**

Correct as it follows the necessary steps of applying polynomial transformations to the training data, fitting the logistic regression model on this transformed data, and using the same transformation logic on the test data before making predictions. This ensures the model is trained and tested on the same type of data, which is crucial for accurate model evaluation.

-

**`model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)`**

Does not apply polynomial transformations to the data.

-

**`model = LogisticRegression()
model.fit(X_test_poly, y_test)
y_pred = model.predict(X_train_poly)`**

Incorrectly uses test data for model training.

-

```
model = LogisticRegression()  
model.fit(X_train_poly, y_test)  
y_pred = model.predict(X_test_poly)
```

Incorrectly uses the test set labels for training the model.